Software Engineering

UML Use Case Diagramm

Prof. Dr.-Ing. Sven Graupner



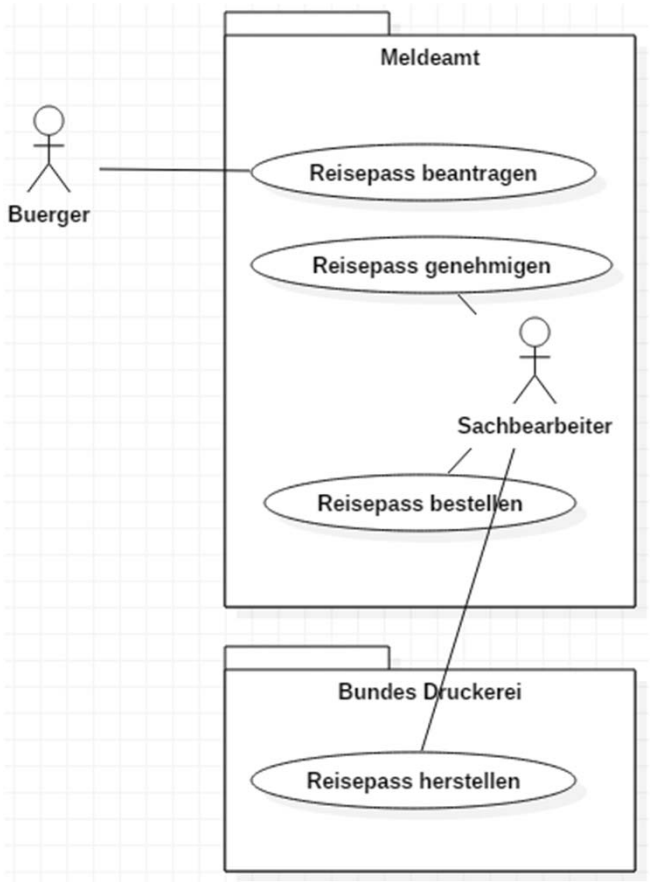
UML Use Case Diagramm

UML Use Case Diagramm erlaubt eine übersichtliche, vereinfachte graphische Darstellung von **Anwendungs-fällen** mit:

- **Akteuren** und ihren Anwendungsfällen.
- **Begrenzungen** (Boundaries).

Weiterhin gibt es:

- Beziehungen zwischen Use Cases <<include>>, <<extend>>, Generalisierung.
- Beziehungen zwischen Use Case Diagrammen mit “extension points“.

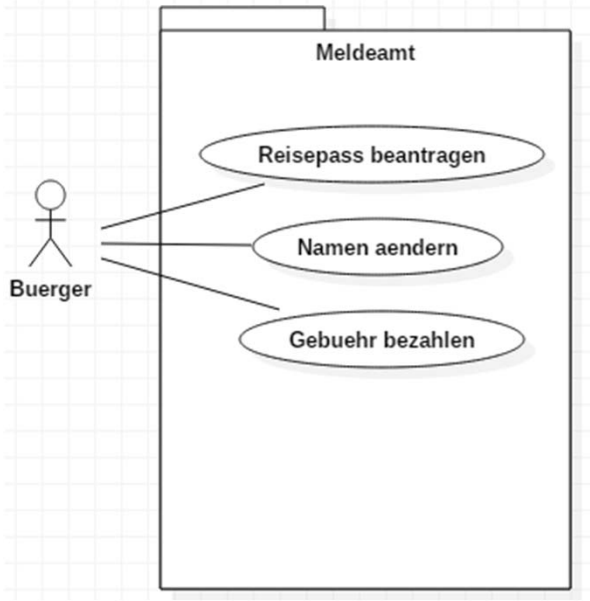


https://www.uml-diagrams.org/use-case-diagrams.html

Zweck von Use Case Diagrammen

UML Use Case Diagramme erlauben einen Überblick, jedoch keine vollständige Beschreibung von Use Cases:

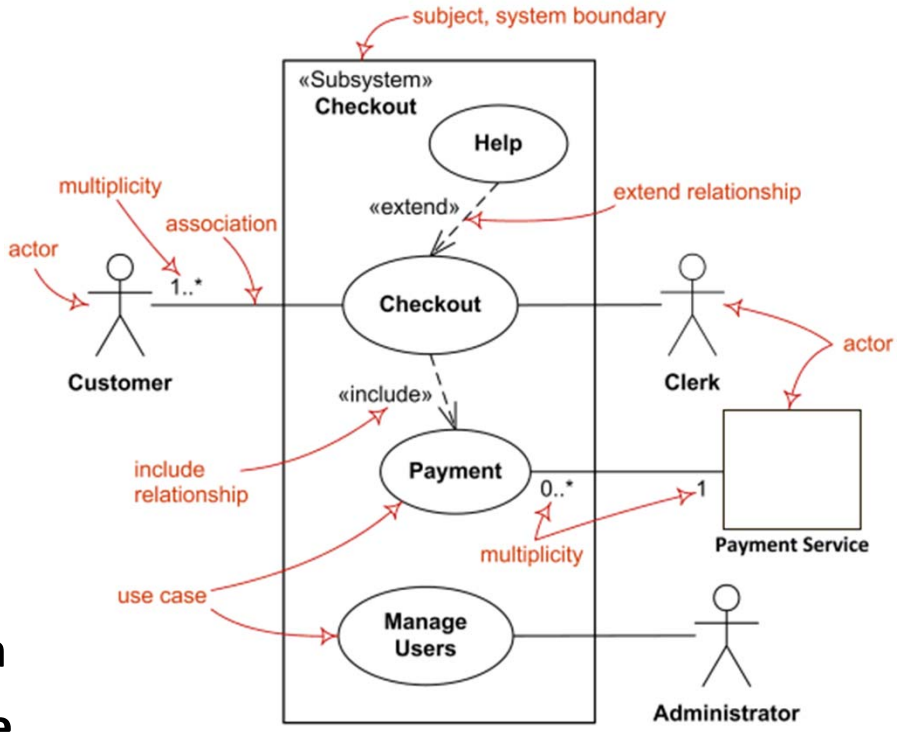
- Keine Beschreibung des Ablaufs der Szenarien.
- Keine Beschreibung von Auslöser- und Vorbedingungen.
- Keine Beschreibung von Erfolgs- oder Fehler-zuständen.



UML System Use Case Diagramm (für Systeme)

UML unterscheidet zwischen System Use Cases und Business Use Cases.

- Am häufigsten sind Use Cases mit Systemen, die durch einen Rahmen (system boundary) abgegrenzt sind und einen Namen haben, im Bsp. **<<Subsystem>> Checkout** .
- Akteure sind stets außerhalb des Systems und repräsentieren Rollen von Individuen bzw. von externen Systemen, wie dem **Payment Service** im Bsp.

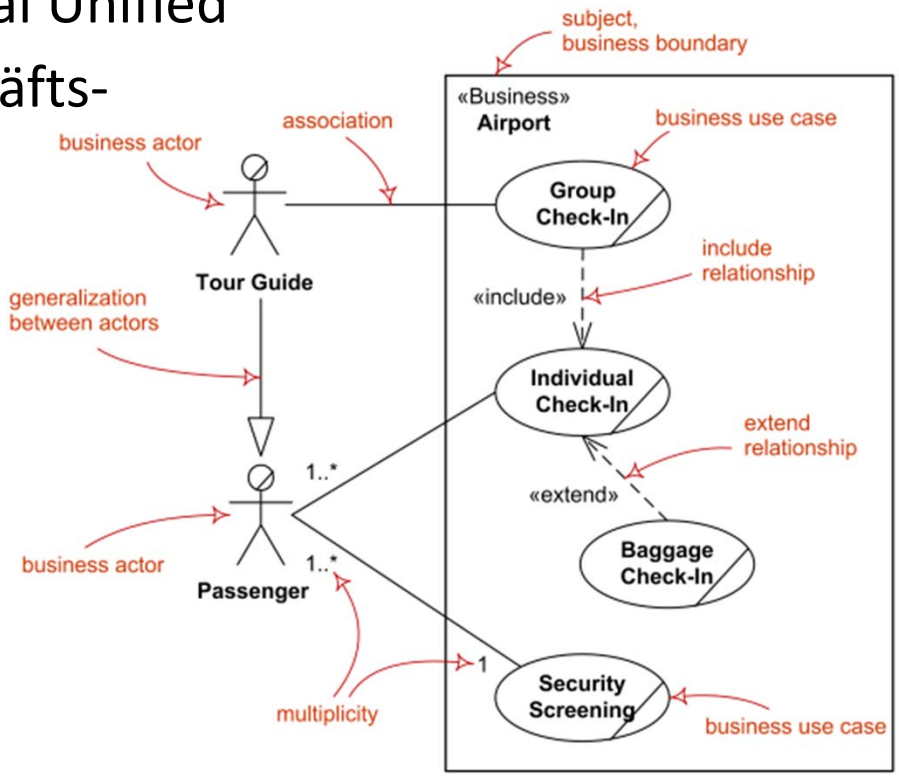


https://www.uml-diagrams.org/use-case-diagrams.html

UML Use Case Diagramm (für Geschäftsfälle)

Business Use Cases wurde von Rational Unified Process RUP zur Modellierung von Geschäftsfällen übernommen.

- Boundaries sind hier Begrenzungen von organisatorischen Einheiten, z.B. von Abteilungen oder Ämtern, und nicht von technischen Systemen.
- Business Use Cases können vollständig zwischen Personen bzw. zwischen Organisationen ausgeführt werden.

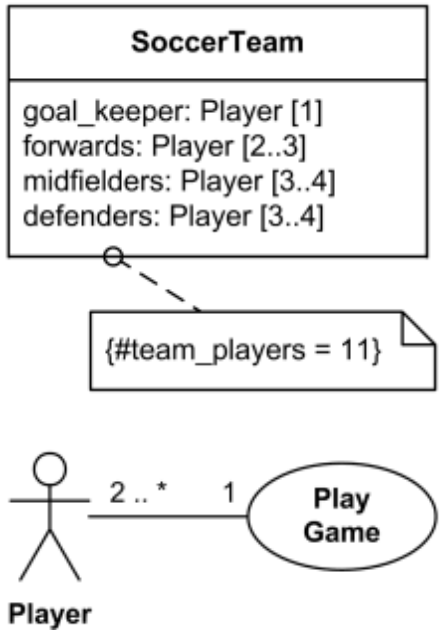


https://www.uml-diagrams.org/use-case-diagrams.html

Kardinalitäten (Multiplicities)

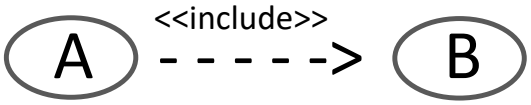
Kardinalitäten (multiplicities) erlauben, Anzahlen auszudrücken.

Multiplicity	Option	Cardinality
0..0	0	Collection must be empty
0..1		No instances or one instance
1..1	1	Exactly one instance
0..*	*	Zero or more instances
1..*		At least one instance
5..5	5	Exactly 5 instances
m..n		At least m but no more than n instances



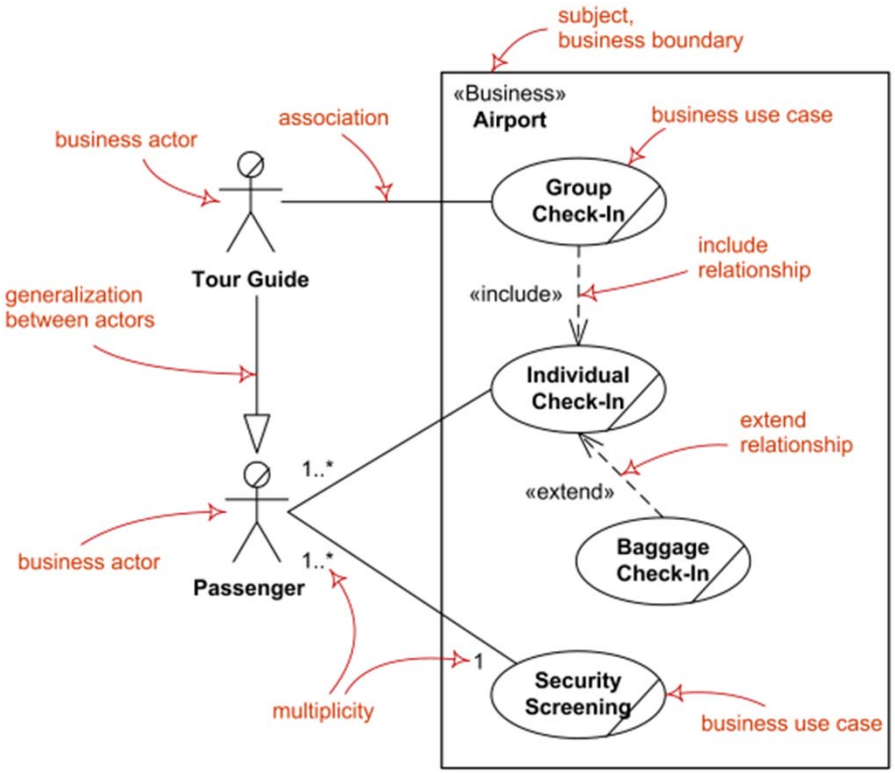
https://www.uml-diagrams.org/multiplicity.html

Die <<include>>-Beziehung



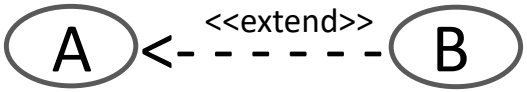
bedeutet, dass ein Use Case A einen Use Case B enthält. Tritt Use Case A ein, ist Use Case B **stets** mit **eingeschlossen** (A ohne B geht nicht).

- Im Bsp. enthält der Use Case „Group Check-In“ den Use Case „Individual Check-In“, da ein Gruppen Check-in aus den Einzel-Checkin‘s der Gruppenmitglieder besteht.



https://www.uml-diagrams.org/use-case-include.html

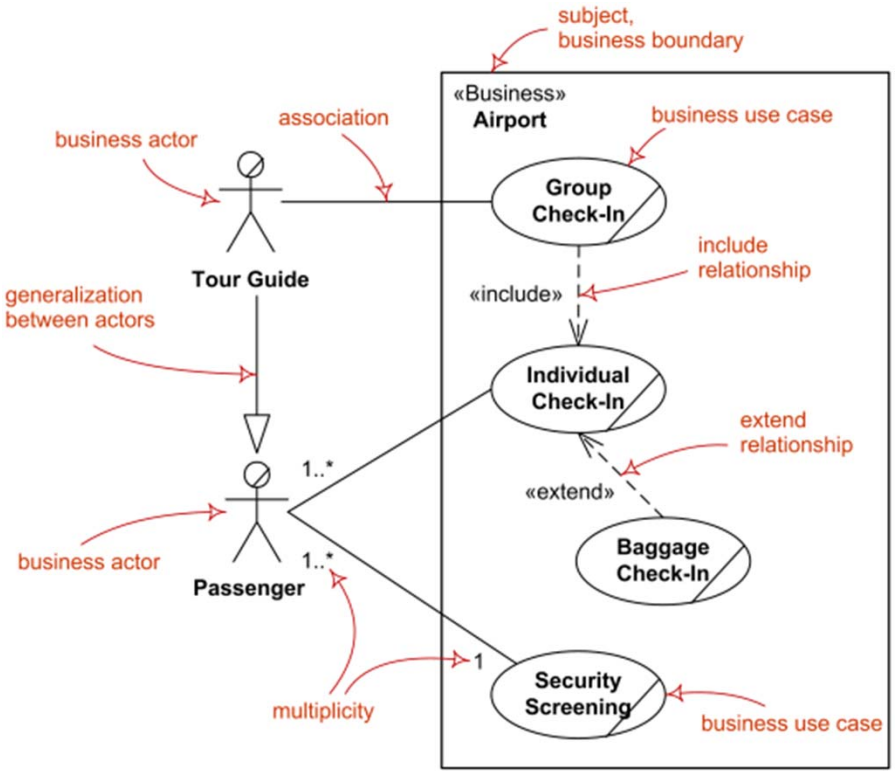
Die <<extend>>-Beziehung



bedeutet, dass Use Case B alle Eigenschaften von Use Case A **enthält und diese erweitert**. A kann unabhängig von B existieren.

B ist eine Variante von A.

- Im Bsp. erweitert „Baggage Check-In“ (mit Gepäck) den allgemeinen Use Case „Individual Check-In“ (mit und ohne Gepäck).



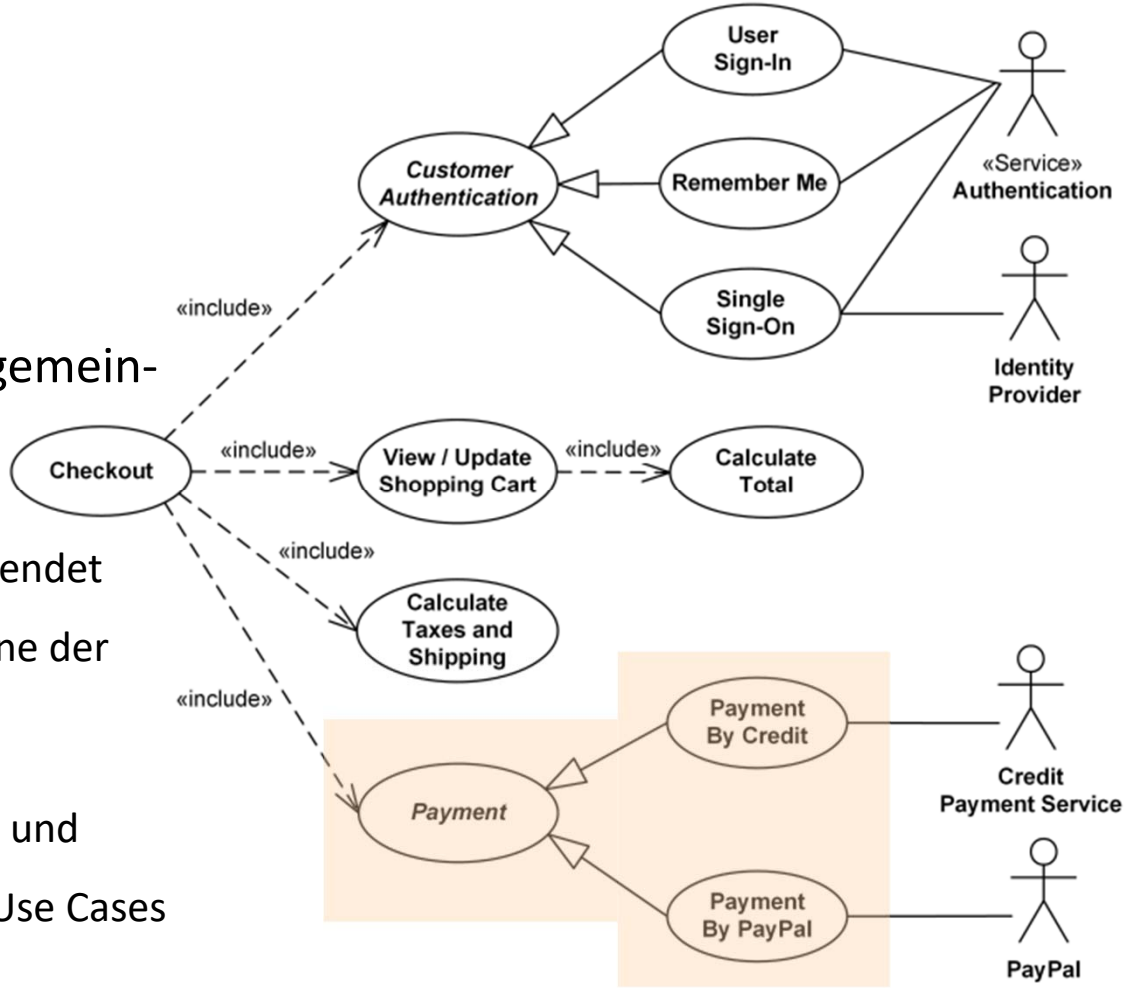
https://www.uml-diagrams.org/use-case-extend.html

Die Generalisierung



bedeutet, dass Use Case A eine Verallgemeinerung von Use Case B (*is_a*) ist.

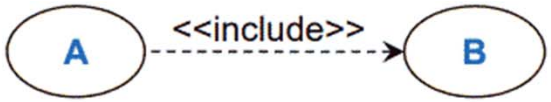
- Während <<extend>> für Alternativen verwendet wird, schließt die Generalisierung jeweils eine der abgeleiteten Fälle ein.
- Im Bsp. sind Use Cases „Payment by Credit“ und „Payment by PayPal“ spezielle Formen des Use Cases „Payment“.



<<include>>, <<extend>>, Generalisierung

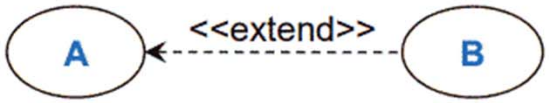
Include-Beziehung

- Immer wenn A ausgeführt wird, muss auch B ausgeführt werden
- B ist unbedingt nötig, um A auszuführen
- B stellt Verhalten dar, das von mehreren Use Cases oder von einem Akteur direkt genutzt wird



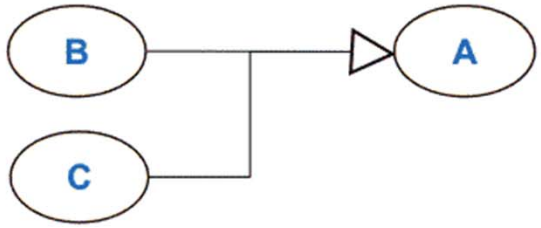
Extend-Beziehung

- Wenn A ausgeführt wird, kann auch B ausgeführt werden
- B ist nicht zwingend nötig, um A auszuführen
- B stellt Sonder- oder Fehelrfälle, optionales oder seltenes Verhalten dar



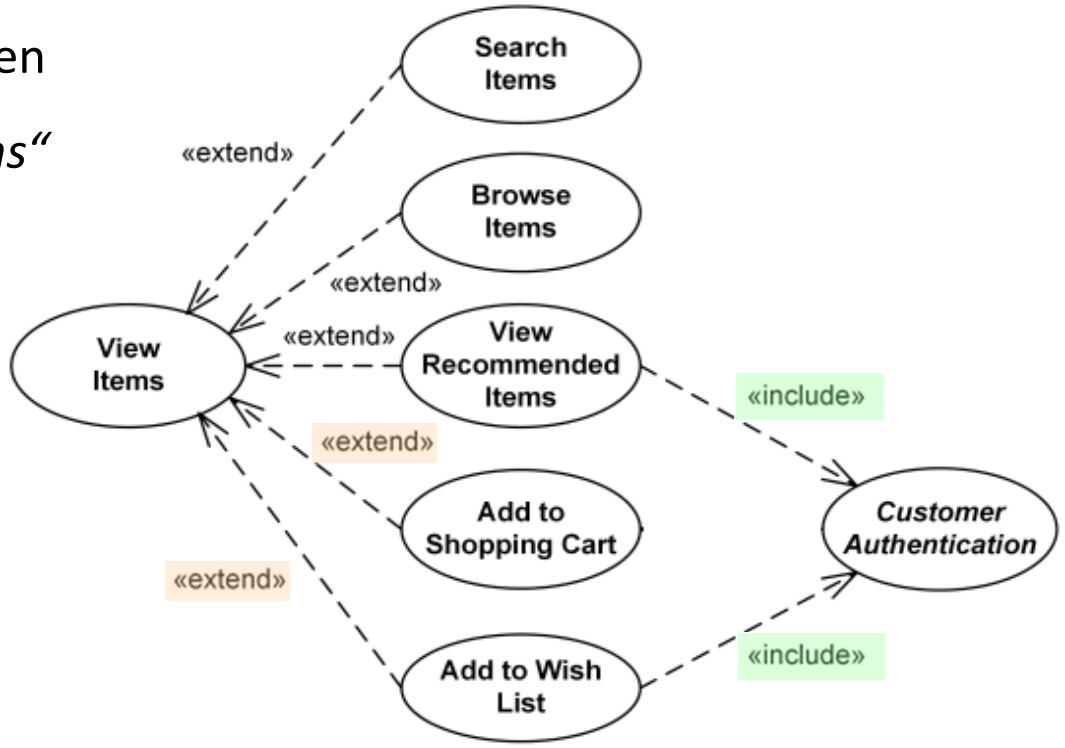
Generalisierungs-Beziehung

- B und C sind jeweils spezielle Ausprägungen des gesamten Ablaufs von A
- Nur eine der verschiedenen Spezialisierungen wird durchgeführt, die aber komplett

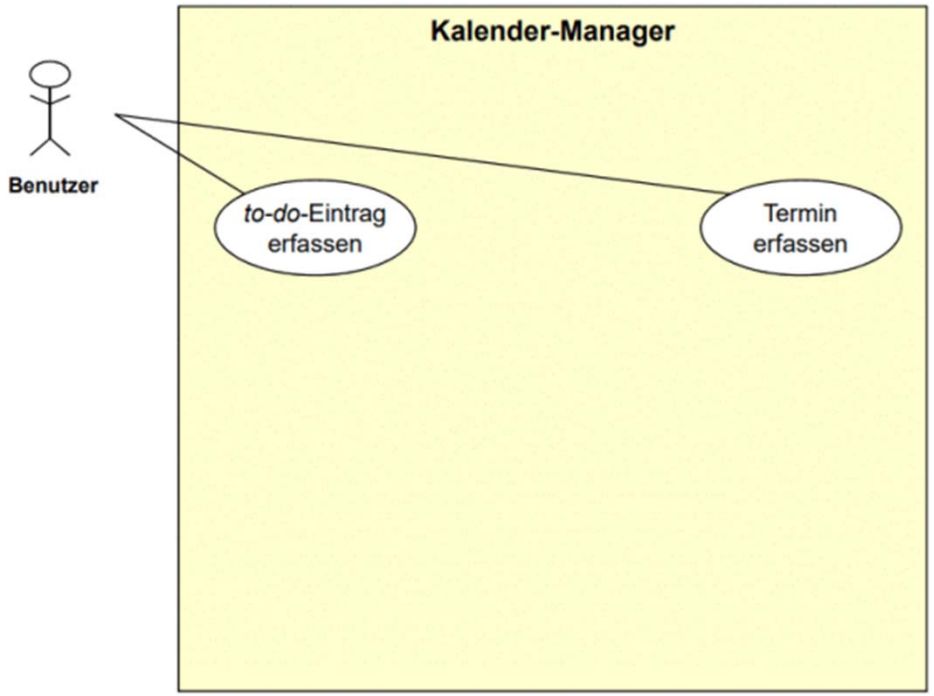


Beispiel 1: Shopping

- *“Customer Authentication“* ist in beiden Use Cases *“View Re-commended Items“* und “*Add to Wish List“* enthalten.
- Use Cases *“Add to Shopping Cart“* und *“Add to Wish List“* erweitern den Use Case “*View Items“.*

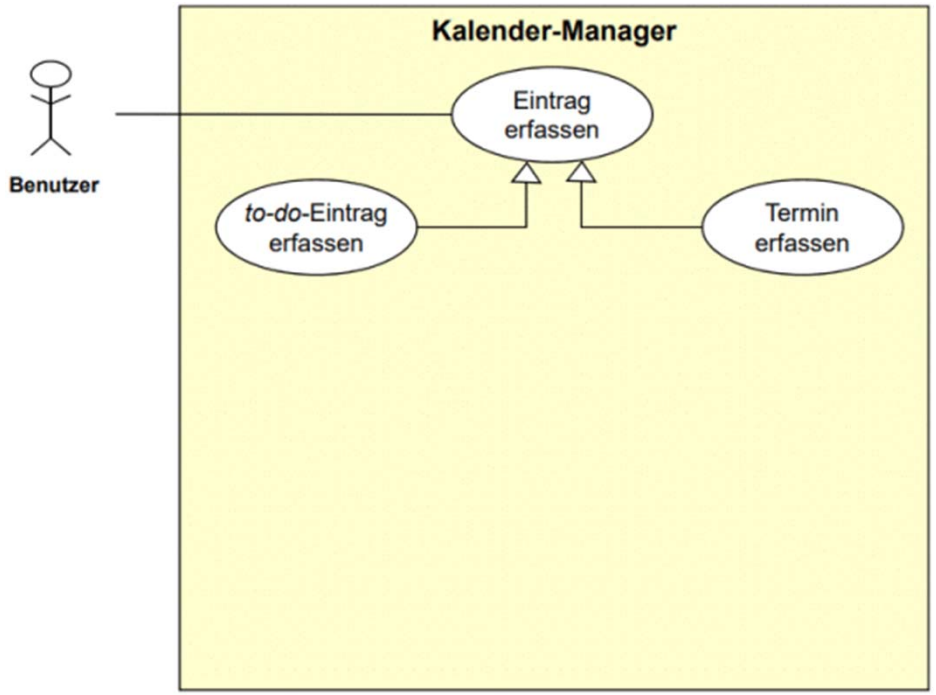


Beispiel 2: Kalender: Begrenzung, Haupt-Use Cases



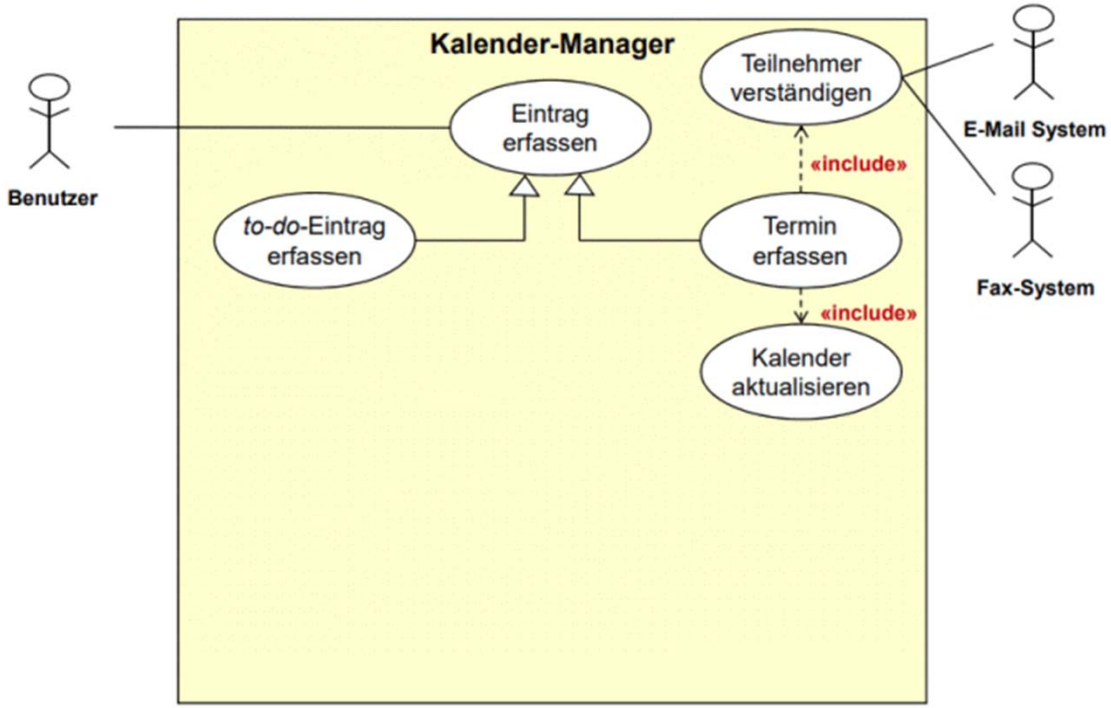
- Die Systembegrenzung ist ein Kalender-Manager.
- Der Akteur Benutzer hat zwei wesentliche Use Cases für den Kalender: „to-do-Eintrag erfassen“ und „Termin erfassen“.

Beispiel 2: Kalender: Generalisierung



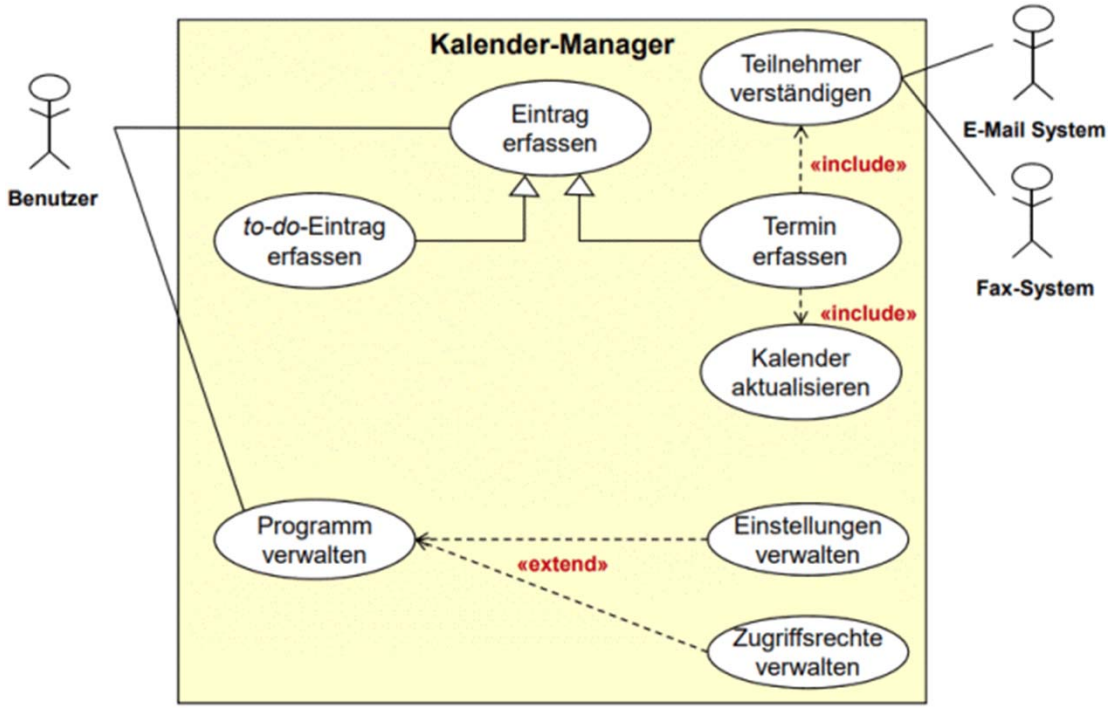
- Die beiden Use Cases „to-do-Eintrag erfassen“ und „Termin erfassen“ können zu „Eintrag erfassen“ verallgemeinert (generalisiert) werden.
- Mit der Generalisierung lassen sich gemeinsame Eigenschaften im allgemeineren Use Case beschreiben und jeweils spezielle Eigenschaften in den untergeordneten Use Cases.

Beispiel 2: Kalender: <<include>>



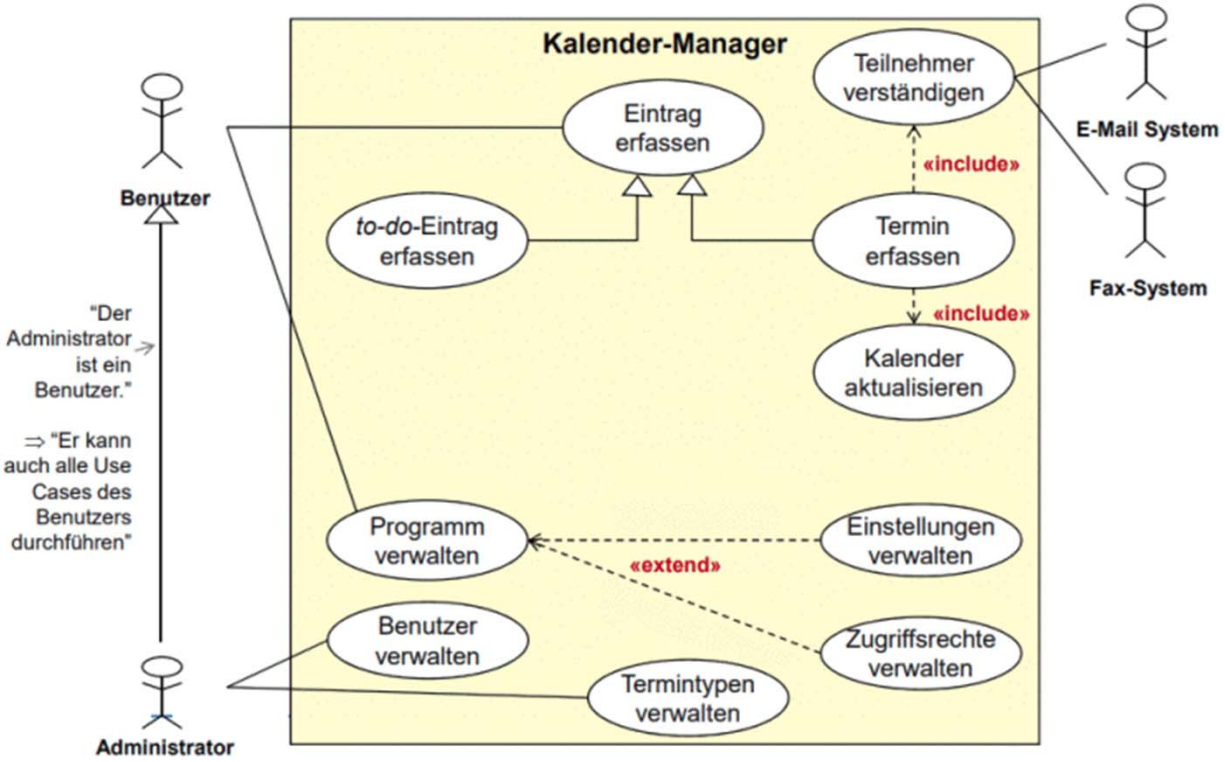
- Der Use Cases „Termin erfassen“ schließt Use Cases „Teilnehmer verständigen“ und „Kalender aktualisieren“ ein.
- Use Case „Teilnehmer verständigen“ betrifft weitere Akteure: E-Mail System und Fax-System.

Beispiel 2: Kalender: <<extend>>



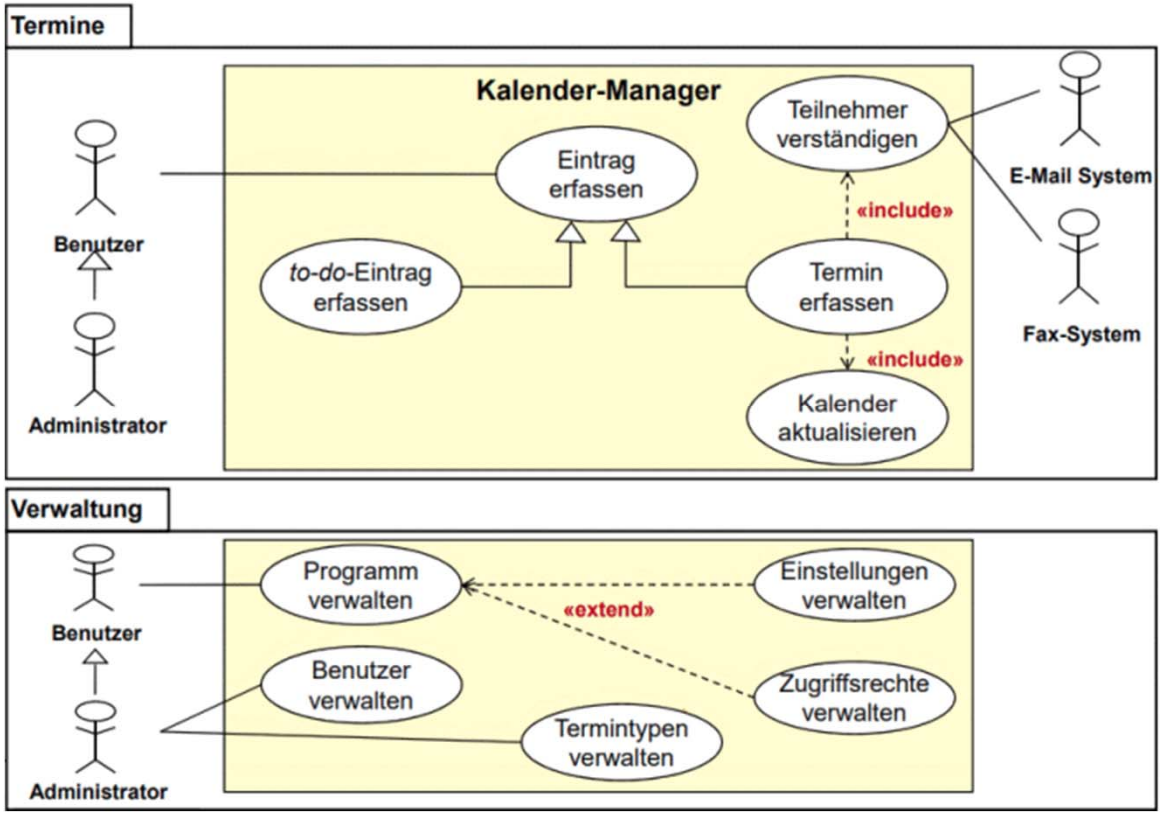
- Der Akteur Benutzer bekommt einen weiteren Use Case: „Programm verwalten“.
- Dieser Use Case erweitert Use Cases „Einstellungen verwalten“ und „Zugriffsrechte verwalten“.

Beispiel 2: Kalender: Rollen



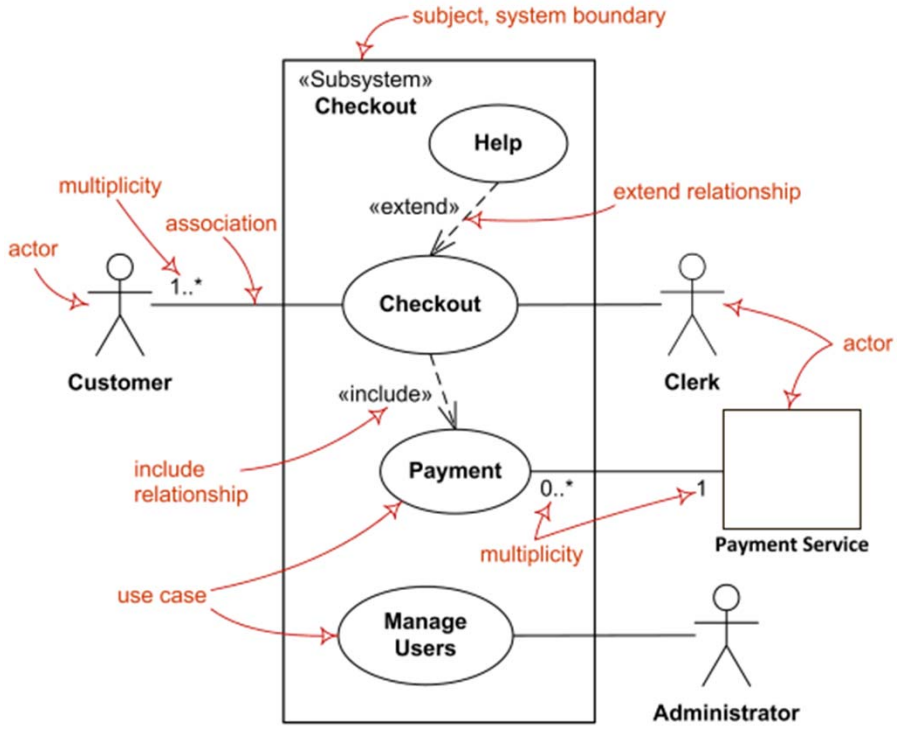
- Aufgaben „Benutzer verwalten“ sowie „Termintypen verwalten“ werden einem weiteren Akteur Administrator zugeordnet, der auch ein Benutzer ist.
- Die Generalisierungs-Relation zwischen Administrator und Benutzer wird auch als „ist ein“ (is_a)-Relation bezeichnet.

Beispiel 2: Kalender: Systemaufteilung



- Zum Schluss kann man die Aufgaben des Kalenders noch von den Aufgaben der Programmverwaltung in zwei separate Bereiche trennen.

Zusammenfassung: Use Case Diagramm



https://www.uml-diagrams.org/use-case-diagrams.html